

Big Data Analytics Lab Project

Analyzing Student Academic Performance using Apache Pig and Hadoop MapReduce

Submitted by: Abir

1. Introduction	3
2. Dataset Description	3
Dataset Schema	3
Key Characteristics of the Dataset	4
3. MapReduce Analysis	5
3.1 Average GPA by School Region	5
3.2 Pass Rate by School Type	6
3.3 Average Stress and Motivation by Parental Involvement	7
4. Pig Analysis	9
4.1 Average GPA by School Region	9
4.2 Study Hours vs Stress Level	9
4.3 Top 10 Students by GPA	10
4.4 Attendance and GPA by Laptop Access	11
4.5 Average Subject Scores by School Type and Gender	11
5. Conclusion	13

1. Introduction

This project focuses on analyzing a student academic performance dataset using Apache Pig and Hadoop MapReduce on top of the Hadoop ecosystem. The goal is to understand how demographic factors, study habits, and school environment relate to student outcomes such as GPA and pass/fail status.

At the core of this system is Hadoop, which provides a distributed framework for storing and processing big data. Hadoop mainly consists of two key components used in this project:

HDFS (Hadoop Distributed File System)

HDFS is responsible for storing the dataset across the system. Instead of keeping data in a single machine, it distributes the file into blocks and stores them across nodes, ensuring scalability and fault tolerance. The dataset (students.csv) is first placed into HDFS so that it can be accessed efficiently by both Pig and MapReduce jobs.

MapReduce Framework

MapReduce is the processing layer of Hadoop. It divides large data into smaller chunks, processes them in parallel using mappers, and then aggregates results using reducers, allowing efficient handling of thousands of records.

On top of Hadoop, Apache Pig is used as a higher-level abstraction. Pig scripts are automatically converted into MapReduce jobs internally, which means both approaches ultimately run on the same distributed infrastructure (HDFS + MapReduce engine).

The project performs multiple analytical operations on the dataset to extract meaningful insights about student performance. This includes finding average GPA across school regions, calculating pass rates by school type, studying the relationship between parental involvement and student stress/motivation, checking how study hours relate to stress levels, identifying top-performing students, examining the effect of laptop access on attendance and GPA, and comparing subject scores across school types and gender.

2. Dataset Description

The dataset used in this project (students.csv) was obtained from Kaggle. It contains 10,000 records, each representing an individual student, along with demographic information, study habits, wellbeing indicators, and academic outcomes.

Dataset Schema

Field Name	Data Type	Description
student_id	String	Unique identifier for each student
age	Integer	Age of the student
gender	String	Gender of the student (Male / Female / Non-binary)
ethnicity	String	Ethnicity category, coded as A–D
parental_education	String	Highest education level of a parent (e.g. bachelor, phd)
family_income	String	Family income bracket (low / middle / high)
school_type	String	Type of school (public / private / charter)
school_region	String	Region where the school is located (urban / suburban / rural)
study_hours_per_week	Float	Hours spent studying per week
attendance_rate	Float	Attendance percentage

extracurricular_activities	Integer	Number of extracurricular activities
sports_participation	String	Whether the student plays sports (yes / no)
tutoring_sessions	Integer	Number of tutoring sessions attended
parental_involvement	String	Level of parental involvement (low / medium / high)
internet_access	String	Whether the student has internet access (yes / no)
has_laptop	String	Whether the student owns a laptop (yes / no)
sleep_hours	Float	Average hours of sleep per night
stress_level	Float	Self-reported stress level (0–10 scale)
motivation_score	Float	Self-reported motivation score (0–10 scale)
reading_score	Float	Reading test score
writing_score	Float	Writing test score
math_score	Float	Math test score
science_score	Float	Science test score
overall_gpa	Float	Overall GPA (0–4 scale)
passed	Integer	Pass / fail flag (1 = passed, 0 = failed)

Key Characteristics of the Dataset

- Scale: 10,000 rows, suitable for demonstrating distributed processing with Pig and MapReduce.
- Rich attribute set: covers demographics, family background, school environment, study habits, wellbeing, and academic scores.
- Balanced categorical fields: school_type, school_region, and parental_involvement each have a small number of well-populated categories, making them suitable grouping keys.
- Numeric outcome fields: overall_gpa and the four subject scores allow averaging and ranking analyses.
- Binary outcome field: 'passed' allows pass-rate calculations across groups.

3. MapReduce Analysis

Three MapReduce jobs were implemented in Java using the Hadoop MapReduce API, each following the standard Driver–Mapper–Reducer pattern.

3.1 Average GPA by School Region

Objective:

Returns the average overall GPA for students grouped by school region (urban, suburban, rural).

This analysis comprises of:

- Driver Class: Sets up job configuration, mapper and reducer classes, and defines output types.
- Mapper Class: Emits school_region (Text) as key and overall_gpa (DoubleWritable) as value.
- Reducer Class: Calculates the sum and count of GPA values per region, then outputs the average.

Code Snippet:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AvgGpaByRegion {

    public static class Map extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
            InterruptedException {
            String[] f = value.toString().split(",");
            if (f.length < 25 || f[0].equals("student_id")) return;
            outKey.set(f[7]); // school_region
            context.write(outKey, new DoubleWritable(Double.parseDouble(f[23]))); // overall_gpa
        }
    }

    public static class Reduce extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
        public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
            throws IOException, InterruptedException {
            double sum = 0;
            int count = 0;
            for (DoubleWritable v : values) {
                sum += v.get();
                count++;
            }
            context.write(key, new DoubleWritable(sum / count));
        }
    }

    public static void main(String[] args) throws Exception {
        Job job = Job.getInstance(new Configuration(), "Avg GPA By Region");
        job.setJarByClass(AvgGpaByRegion.class);
        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
    }
}
```

```

        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Output :

```

regionavg_gpastudent_count
urban3.1303967
suburban3.1254008
rural3.1092025

```

3.2 Pass Rate by School Type

Objective:

Calculates what percentage of students passed, grouped by school type (public, private, charter).

This analysis comprises of:

- Driver Class: Configures mapper output (Text, IntWritable) and reducer output (Text, Text).
- Mapper Class: Emits school_type as key and the passed flag (0/1) as value.
- Reducer Class: Sums the passed flags and the total count, then computes the pass percentage.

Code Snippet:

```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PassRateBySchoolType {

    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
            InterruptedException {
            String[] f = value.toString().split(",");
            if (f.length < 25 || f[0].equals("student_id")) return;
            outKey.set(f[6]); // school_type
            context.write(outKey, new IntWritable(Integer.parseInt(f[24]))); // passed (0/1)
        }
    }

    public static class Reduce extends Reducer<Text, IntWritable, Text, Text> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
            int passed = 0;
            int total = 0;
            for (IntWritable v : values) {
                passed += v.get();
                total++;
            }
            double rate = (passed * 100.0) / total;
            context.write(key, new Text(String.format("%.2f%% (n=%d)", rate, total)));
        }
    }

    public static void main(String[] args) throws Exception {
        Job job = Job.getInstance(new Configuration(), "Pass Rate By School Type");
    }
}

```

```

        job.setJarByClass(PassRateBySchoolType.class);
        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Output :

```

school_typepass_rate (count)
private97.62%2021
charter96.29%1455
public95.59%6524

```

3.3 Average Stress and Motivation by Parental Involvement

Objective:

Examines how student stress level and motivation score vary with the level of parental involvement (low, medium, high).

This analysis comprises of:

- Driver Class: Sets up job configuration for a (Text, Text) mapper/reducer pipeline.
- Mapper Class: Emits parental_involvement as key and a combined 'stress,motivation' string as value.
- Reducer Class: Parses each value pair, accumulates sums, and outputs the average stress and motivation per group.

Code Snippet:

```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class StressMotivationByParentalInvolvement {

    public static class Map extends Mapper<LongWritable, Text, Text, Text> {
        private Text outKey = new Text();
        private Text outVal = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
            String[] f = value.toString().split(",");
            if (f.length < 25 || f[0].equals("student_id")) return;
            outKey.set(f[13]); // parental_involvement
            outVal.set(f[17] + "," + f[18]); // stress_level,motivation_score
            context.write(outKey, outVal);
        }
    }

    public static class Reduce extends Reducer<Text, Text, Text, Text> {
        public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {

```

```

        double stressSum = 0, motiSum = 0;
        int count = 0;
        for (Text v : values) {
            String[] parts = v.toString().split(",");
            stressSum += Double.parseDouble(parts[0]);
            motiSum += Double.parseDouble(parts[1]);
            count++;
        }
        String result = String.format("avg_stress=%.3f, avg_motivation=%.3f, count=%d",
            stressSum / count, motiSum / count, count);
        context.write(key, new Text(result));
    }
}

public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "Stress Motivation By Parental Involvement");
    job.setJarByClass(StressMotivationByParentalInvolvement.class);
    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Output:

```

parental_involvementresult
highavg_stress=5.261, avg_motivation=6.132, count=3036
mediumavg_stress=5.242, avg_motivation=5.475, count=4460
lowavg_stress=5.206, avg_motivation=4.878, count=2504

```

4. Pig Analysis

The same dataset was also analyzed using Apache Pig, which expresses the same logic in fewer, more declarative lines of Pig Latin.

4.1 Average GPA by School Region

- Students are grouped by school_region.
- The average overall_gpa and student count are computed for each region and sorted in descending order of GPA.

Code Snippet:

```
students = LOAD 'students.csv' USING PigStorage(',')
  AS (student_id:chararray, age:int, gender:chararray, ethnicity:chararray,
    parental_education:chararray, family_income:chararray, school_type:chararray,
    school_region:chararray, study_hours_per_week:float, attendance_rate:float,
    extracurricular_activities:int, sports_participation:chararray,
    tutoring_sessions:int, parental_involvement:chararray, internet_access:chararray,
    has_laptop:chararray, sleep_hours:float, stress_level:float, motivation_score:float,
    reading_score:float, writing_score:float, math_score:float, science_score:float,
    overall_gpa:float, passed:int);

students = FILTER students BY student_id != 'student_id';

grouped_by_region = GROUP students BY school_region;

avg_gpa_by_region = FOREACH grouped_by_region GENERATE
  group AS school_region,
  AVG(students.overall_gpa) AS avg_gpa,
  COUNT(students) AS student_count;

avg_gpa_by_region = ORDER avg_gpa_by_region BY avg_gpa DESC;

DUMP avg_gpa_by_region;
STORE avg_gpa_by_region INTO 'output/avg_gpa_by_region' USING PigStorage(',');
```

Output:

```
school_regionavg_gpastudent_count
urban3.1303967
suburban3.1254008
rural3.1092025
```

4.2 Study Hours vs Stress Level

- Students are bucketed into four study-hour ranges (low, medium, high, very high).
- Average stress level and motivation score are computed for each bucket to see if more study hours relate to higher stress.

Code Snippet:

```
students = LOAD 'students.csv' USING PigStorage(',')
  AS (student_id:chararray, age:int, gender:chararray, ethnicity:chararray,
    parental_education:chararray, family_income:chararray, school_type:chararray,
    school_region:chararray, study_hours_per_week:float, attendance_rate:float,
    extracurricular_activities:int, sports_participation:chararray,
    tutoring_sessions:int, parental_involvement:chararray, internet_access:chararray,
    has_laptop:chararray, sleep_hours:float, stress_level:float, motivation_score:float,
    reading_score:float, writing_score:float, math_score:float, science_score:float,
    overall_gpa:float, passed:int);
```

```

students = FILTER students BY student_id != 'student_id';

bucketed = FOREACH students GENERATE
(
    (study_hours_per_week < 5) ? 'low (<5h)' :
    (study_hours_per_week < 10) ? 'medium (5-10h)' :
    (study_hours_per_week < 15) ? 'high (10-15h)' : 'very_high (15h+)'
) AS study_bucket,
stress_level,
motivation_score;

grouped_by_bucket = GROUP bucketed BY study_bucket;

stress_by_bucket = FOREACH grouped_by_bucket GENERATE
    group AS study_bucket,
    AVG(bucketed.stress_level) AS avg_stress,
    AVG(bucketed.motivation_score) AS avg_motivation,
    COUNT(bucketed) AS student_count;

stress_by_bucket = ORDER stress_by_bucket BY study_bucket;

DUMP stress_by_bucket;
STORE stress_by_bucket INTO 'output/study_hours_vs_stress' USING PigStorage(',');

```

Output :

```

study_bucketavg_stressavg_motivationstudent_count
low (<5h)4.9065.6051607
medium (5-10h)5.1305.4702639
high (10-15h)5.2495.4923101
very_high (15h+)5.5385.5702653

```

4.3 Top 10 Students by GPA

- Students are ranked by overall_gpa in descending order.
- The top 10 highest-performing students are selected, along with their school type, region, attendance rate, and study hours.

Code Snippet:

```

students = LOAD 'students.csv' USING PigStorage(',')
    AS (student_id:chararray, age:int, gender:chararray, ethnicity:chararray,
        parental_education:chararray, family_income:chararray, school_type:chararray,
        school_region:chararray, study_hours_per_week:float, attendance_rate:float,
        extracurricular_activities:int, sports_participation:chararray,
        tutoring_sessions:int, parental_involvement:chararray, internet_access:chararray,
        has_laptop:chararray, sleep_hours:float, stress_level:float, motivation_score:float,
        reading_score:float, writing_score:float, math_score:float, science_score:float,
        overall_gpa:float, passed:int);

students = FILTER students BY student_id != 'student_id';

relevant = FOREACH students GENERATE
    student_id, school_type, school_region, overall_gpa, attendance_rate, study_hours_per_week;

ranked = ORDER relevant BY overall_gpa DESC;
top10 = LIMIT ranked 10;

DUMP top10;
STORE top10 INTO 'output/top10_students_by_gpa' USING PigStorage(',');

```

Output :

```

student_id,school_type,school_region,gpa,attendance_rate,study_hours
STU00082publicurban4.0100.019.7
STU00180publicurban4.085.320.1
STU00225chartersuburban4.071.419.0
STU00238publicurban4.092.99.1
STU00240publicurban4.073.68.2
STU00259privatesuburban4.093.313.5
STU00282publicsuburban4.071.021.1
STU00284charterrural4.084.822.8
STU00307publicurban4.093.08.3
STU00334charterurban4.0100.019.0

```

4.4 Attendance and GPA by Laptop Access

- Students are grouped by whether they own a laptop (has_laptop = yes/no).
- Average attendance rate and average GPA are compared between the two groups.

Code Snippet:

```

students = LOAD 'students.csv' USING PigStorage(',')
AS (student_id:chararray, age:int, gender:chararray, ethnicity:chararray,
    parental_education:chararray, family_income:chararray, school_type:chararray,
    school_region:chararray, study_hours_per_week:float, attendance_rate:float,
    extracurricular_activities:int, sports_participation:chararray,
    tutoring_sessions:int, parental_involvement:chararray, internet_access:chararray,
    has_laptop:chararray, sleep_hours:float, stress_level:float, motivation_score:float,
    reading_score:float, writing_score:float, math_score:float, science_score:float,
    overall_gpa:float, passed:int);

students = FILTER students BY student_id != 'student_id';

grouped_by_laptop = GROUP students BY has_laptop;

attendance_by_laptop = FOREACH grouped_by_laptop GENERATE
    group AS has_laptop,
    AVG(students.attendance_rate) AS avg_attendance,
    AVG(students.overall_gpa) AS avg_gpa,
    COUNT(students) AS student_count;

attendance_by_laptop = ORDER attendance_by_laptop BY has_laptop;

DUMP attendance_by_laptop;
STORE attendance_by_laptop INTO 'output/attendance_by_laptop_access' USING PigStorage(',');

```

Output :

```

has_laptop,avg_attendance,avg_gpa,student_count
yes,82.9523,1587743
no,83.3643,0052257

```

4.5 Average Subject Scores by School Type and Gender

- Students are grouped by the combination of school_type and gender.
- Average reading, writing, math, and science scores are computed for each group.

Code Snippet:

```

students = LOAD 'students.csv' USING PigStorage(',')
AS (student_id:chararray, age:int, gender:chararray, ethnicity:chararray,
    parental_education:chararray, family_income:chararray, school_type:chararray,
    school_region:chararray, study_hours_per_week:float, attendance_rate:float,
    extracurricular_activities:int, sports_participation:chararray,

```

```

        tutoring_sessions:int, parental_involvement:chararray, internet_access:chararray,
        has_laptop:chararray, sleep_hours:float, stress_level:float, motivation_score:float,
        reading_score:float, writing_score:float, math_score:float, science_score:float,
        overall_gpa:float, passed:int);

students = FILTER students BY student_id != 'student_id';

grouped = GROUP students BY (school_type, gender);

avg_scores = FOREACH grouped GENERATE
    FLATTEN(group) AS (school_type, gender),
    AVG(students.reading_score) AS avg_reading,
    AVG(students.writing_score) AS avg_writing,
    AVG(students.math_score) AS avg_math,
    AVG(students.science_score) AS avg_science,
    COUNT(students) AS student_count;

avg_scores = ORDER avg_scores BY school_type, gender;

DUMP avg_scores;
STORE avg_scores INTO 'output/avg_scores_by_schooltype_gender' USING PigStorage(',');

```

Output :

```

school_typegenderavg_readingavg_writingavg_mathavg_sciencelcount
charterF77.6377.3680.4079.89693
charterM78.1177.4280.4979.16702
charterNon-binary75.2776.0780.7078.7260
privateF79.1178.6481.1180.55953
privateM79.5879.9082.2781.49987
privateNon-binary80.9378.9782.9981.6481
publicF76.5376.3679.1278.343190
publicM76.1175.9678.9277.993080
publicNon-binary76.8876.2079.4679.02254

```

5. Conclusion

This project demonstrates how a moderately large student performance dataset can be processed and analyzed using both Hadoop MapReduce and Apache Pig. Each analytical question — average GPA by region, pass rate by school type, stress and motivation by parental involvement, study hours versus stress, top-performing students, laptop access effects, and subject scores by school type and gender — was implemented and produced consistent results across both approaches.

The results show that GPA is fairly consistent across school regions (roughly 3.11–3.13), private schools have a slightly higher pass rate (97.62%) than public schools (95.59%), and higher parental involvement is associated with noticeably higher average motivation scores (6.13 vs 4.88 for low involvement). Interestingly, laptop access shows only a small difference in average attendance and GPA, suggesting other factors play a larger role in these outcomes.

Pig proved to be more concise and easier to write due to its declarative nature, while MapReduce required more detailed implementation but provided deeper insight into how data flows through the system. Despite the difference in complexity, both approaches produced consistent analytical results.

Overall, the work highlights the importance of choosing the right tool for the use case — Pig is well suited for faster development and simpler workflows, whereas MapReduce offers more flexibility for customized and performance-critical tasks.